

# Roteiro de Apresentação

API de Tarefas com Arquitetura Distribuída — repositório: [github.com/Matheuspol/TrabalhoFinal-API](https://github.com/Matheuspol/TrabalhoFinal-API)

## Checklist antes de sair de casa

- [ ] Docker Desktop testado e funcionando (se for usar notebook próprio)
- [ ] Repositório no GitHub confirmado: [github.com/Matheuspol/TrabalhoFinal-API](https://github.com/Matheuspol/TrabalhoFinal-API)
- [ ] Se for usar PC da sala: chegar cedo. A instalação do Docker pode pedir reinício do Windows (WSL2) — não deixe pra última hora

## Passo 1 — Levar o projeto pro PC (se não for notebook próprio)

```
git clone https://github.com/Matheuspol/TrabalhoFinal-API.git
cd TrabalhoFinal-API
```

## Passo 2 — Instalar Docker Desktop (só se o PC não tiver)

PowerShell como Administrador:

```
winget install --id Docker.DockerDesktop -e
```

Se pedir para instalar/atualizar o WSL:

```
wsl --install
```

Reiniciar o PC → abrir Docker Desktop → aceitar termos → esperar carregar.

## Passo 3 — Subir a aplicação

```
docker compose up --build
```

Sobe 7 containers: MongoDB, RabbitMQ, 2 instâncias da API, gateway e 2 workers. Rode isso assim que chegar na sala, antes da sua vez, pra já estar tudo baixado.

## Passo 4 — Explicar a arquitetura (30 segundos)

*"Temos uma API de tarefas com CRUD, rodando em duas instâncias atrás de um Gateway que faz rate limit e load balancing. As operações da API publicam eventos no RabbitMQ, que dois workers diferentes consomem: um simula notificações, outro faz auditoria."*

## Passo 5 — Demonstrar o CRUD

Tudo via gateway, porta 8080. No PowerShell use sempre **curl.exe** (não só **curl**, que é outro comando).

**Criar tarefa:**

```
curl.exe -s -X POST http://localhost:8080/api/tarefas `
-H "Content-Type: application/json" -H "X-Client-Id: demo" `
-d "{\"titulo\":\"Estudar para a prova\",\"descricao\":\"Revisar RabbitMQ\"}"
```

**Listar (copie o \_id da resposta acima):**

```
curl.exe -s http://localhost:8080/api/tarefas -H "X-Client-Id: demo"
```

**Buscar por id:**

```
curl.exe -s http://localhost:8080/api/tarefas/ID_AQUI -H "X-Client-Id: demo"
```

**Concluir a tarefa:**

```
curl.exe -s -X PUT http://localhost:8080/api/tarefas/ID_AQUI `
-H "Content-Type: application/json" -H "X-Client-Id: demo" -d "{\"concluida\":true}"
```

**Remover:**

```
curl.exe -s -X DELETE http://localhost:8080/api/tarefas/ID_AQUI -H "X-Client-Id: demo"
```

## Passo 6 — Demonstrar Load Balancing

Rode o mesmo GET duas vezes seguidas e mostre o header alternando:

```
curl.exe -s -D - -o NUL -H "X-Client-Id: lb1" http://localhost:8080/api/tarefas | findstr X-Upstream-Instance
curl.exe -s -D - -o NUL -H "X-Client-Id: lb2" http://localhost:8080/api/tarefas | findstr X-Upstream-Instance
```

Uma resposta mostra **todo-api-1**, a outra **todo-api-2** — prova que o gateway está distribuindo entre as duas instâncias.

## Passo 7 — Demonstrar Rate Limit

Dispare 6 chamadas rápidas com o mesmo cliente:

```
1..6 | ForEach-Object {
    curl.exe -s -o NUL -w "%{http_code}`n" -H "X-Client-Id: rl-demo" http://localhost:8080/api/tarefas
}
```

As 5 primeiras respondem **200**, a 6ª responde **429**. Explique: o gateway limita 5 requisições a cada 10 segundos por cliente, identificado pelo header X-Client-Id.

## Passo 8 — Demonstrar o RabbitMQ

Deixe visível o terminal onde rodou **docker compose up** — é lá que os workers imprimem os logs. Crie e conclua uma tarefa de novo (comandos do Passo 5) e aponte os logs aparecendo:

- **worker-notificacoes** só reage a tarefa.criada e tarefa.concluida
- **worker-auditoria** reage a tudo: criada, atualizada, concluida e removida

Opcional (mais visual): abra **http://localhost:15672** no navegador (login guest/guest) e mostre as filas **tarefas.notificacoes.queue** e **tarefas.auditoria.queue** na aba Queues.

## Passo 9 — Encerrar

```
docker compose down
```

## Perguntas técnicas prováveis

|                                                 |                                                                                                                                                          |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Por que 2 instâncias da API?                    | Pra demonstrar load balancing de verdade. As duas usam o mesmo MongoDB, então os dados são compartilhados.                                               |
| Como o gateway decide pra qual instância manda? | Round-robin simples: alterna a cada requisição.                                                                                                          |
| O que acontece se o RabbitMQ cair?              | O CRUD continua funcionando normal, não depende do RabbitMQ pra responder. Só o evento é perdido — isso é tratado com try/catch no código.               |
| Por que duas filas em vez de uma?               | Propósitos diferentes: notificação só liga pra criação/conclusão, auditoria registra tudo. Uma exchange do tipo topic roteia pros lugares certos.        |
| Por que gateway + RabbitMQ, e não Loki?         | Decisão de escopo: Loki exigiria configurar Grafana + Promtail, mais containers e configuração, sem agregar um conceito novo de arquitetura distribuída. |
| Qual stack foi usada?                           | Node.js + Express + MongoDB + RabbitMQ, tudo orquestrado com Docker Compose.                                                                             |